

P1030R0: `std::filesystem::path_view`

Document #: P1030R0
Date: 2018-05-06
Project: Programming Language C++
Library Evolution Working Group
Reply-to: Niall Douglas
<s_sourceforge@nedprod.com>

A proposal for a `std::filesystem::path_view`, a non-owning view of character sequences in the format of a local filesystem path.

A reference implementation of the proposed path view can be found at <https://ned14.github.io/afio/>. It has been found to work well on recent editions of GCC, clang and Microsoft Visual Studio, on x86, x64, ARM and AArch64.

Contents

1	Introduction	2
2	Impact on the Standard	3
3	Proposed Design	3
4	Design decisions, guidelines and rationale	7
4.1	Why on Microsoft Windows interpret chars as UTF-8 when <code>std::filesystem::path</code> interprets chars as ASCII?	7
4.2	Requiring legality of read of character after end of view	9
4.3	Fixed use of stack in <code>struct c_str</code>	9
4.4	Why is only char input supported, except on Microsoft Windows?	10
5	Technical specifications	10
6	Frequently asked questions	10
6.1	Does this mean that all APIs consuming <code>std::filesystem::path</code> ought to now consume <code>std::filesystem::path_view</code> instead?	10
7	Acknowledgements	11
8	References	11

1 Introduction

In the current C++ standard, the canonical way for supplying filesystem paths to C++ functions which consume file system paths is `std::filesystem::path`. This wraps up `std::filesystem::path::string_type = std::basic_string<Char>` with a platform specific choice of `Char` (currently Microsoft Windows uses `Char = wchar_t`, everything else uses `Char = char`) with iterators and member functions which parse the string according to the path delimiters for that platform. For example `std::filesystem::path` on Microsoft Windows might parse this string:

`C:\Windows\System32\notepad.exe`

into:

- `root_name() = "C:"`
- `root_directory() = "\"`
- `root_path() = "C:\"`
- `relative_path() = "Windows\System32\notepad.exe"`
- `parent_path() = "C:\Windows\System32"`
- `filename() = "notepad.exe"`
- `stem() = "notepad"`
- `extension() = ".exe"`
- `*begin() = "C:"`
- `*++begin() = "/"` (note the forward, not backward, slash. This is considered to be the name of the root directory)
- `*++++begin() = "Windows"`
- `*++++++begin() = "System32"` (note no intervening slash)

For every one of these decompositions, a new `path` is returned, which means a new underlying `std::basic_string<Char>`, which means a new memory allocation. In code which performs a lot of path traversal and decomposition, these memory allocations, and the copying of fragments of path around, can start to add up. For example, in [P1031] *Low level file i/o library*, a directory enumeration costs around 250 *nanoseconds* per entry amortised. Each path construction might cost that again. Therefore, for each item enumerated, one *halves* the directory enumeration performance solely due to the choice of `path`, which is why P1031 uses `path_view` instead, and thus can enumerate four million directory items per second which makes handling ten million item plus directories a breeze.

There is also a negative effect on CPU caches of copying around path strings. Paths are increasingly reaching hundred of bytes, as anyone running into the 260 path character limit on Microsoft Windows can testify. Every time one copies a path, one is evicting potentially useful data from the CPU caches, which need not be evicted if one did not copy paths.

Enter thus the proposed `std::filesystem::path_view`, which is a lightweight reference to part, or all of, a source of filesystem path data. It provides most of the same member functions as `std::filesystem::path`, operating by constant and often constexpr reference upon some character source which is in the format of the local platform's file system path, or a generic path, same as with `std::filesystem::path`. It is intended that for most functions currently accepting a `std::filesystem::path`, they can now accept a `std::filesystem::path_view` instead with minor to none refactoring of implementation.

2 Impact on the Standard

The proposed library is a pure-library solution.

3 Proposed Design

Much of the proposed path view is unsurprising, with a large subset of `std::filesystem::path`'s observers and modifiers replicated. Constexpr abounds, and the path view is trivially copyable and is thus suitable for passing around by value.

```
1  class path_view
2  {
3  public:
4      // Const iterator, returns path views of each path section.
5      class const_iterator;
6      // Iterator, aliases const iterator.
7      class iterator;
8      // Const reverse iterator
9      class const_reverse_iterator;
10     // Reverse iterator
11     class reverse_iterator;
12     // Size type
13     using size_type = std::size_t;
14     // Difference type
15     using difference_type = std::ptrdiff_t;
16
17 public:
18
19     // Constructs an empty path view
20     constexpr path_view();
21     ~path_view() = default;
22
23     // Implicitly constructs a path view from a path.
24     // The input path MUST continue to exist for this view to be valid.
25     path_view(const filesystem::path &p) noexcept;
26
27     // Implicitly constructs a path view from a string.
28     // On Windows this is assumed to be in UTF-8, not ASCII.
29     // The input string MUST continue to exist for this view to be valid.
30     path_view(const std::string &s) noexcept;
31
```

```

32 // Implicitly constructs a path view from a zero terminated 'const char *'.
33 // On Windows this is assumed to be in UTF-8, not ASCII.
34 // The input string MUST continue to exist for this view to be valid.
35 constexpr path_view(const char *v) noexcept;
36
37 // Constructs a path view from a lengthed 'const char *'.
38 // On Windows this is assumed to be in UTF-8, not ASCII.
39 // The input string MUST continue to exist for this view to be valid.
40 constexpr path_view(const char *v, size_t len) noexcept;
41
42 /* Implicitly constructs a path view from a string view.
43 On Windows this is assumed to be in UTF-8, not ASCII.
44 \warning The byte after the end of the view must be legal to read.
45 */
46 constexpr path_view(string_view v) noexcept;
47
48 #ifdef _WIN32
49 // Implicitly constructs a path view from a string.
50 // The input string MUST continue to exist for this view to be valid.
51 path_view(const std::wstring &v) noexcept;
52
53 // Implicitly constructs a path view from a zero terminated 'const wchar_t *'.
54 // The input string MUST continue to exist for this view to be valid.
55 constexpr path_view(const wchar_t *v) noexcept;
56
57 // Constructs a path view from a lengthed 'const wchar_t *'.
58 // The input string MUST continue to exist for this view to be valid.
59 constexpr path_view(const wchar_t *v, size_t len) noexcept;
60
61 /* Implicitly constructs a path view from a wide string view.
62 \warning The character after the end of the view must be legal to read.
63 */
64 constexpr path_view(wstring_view v) noexcept;
65 #endif
66
67 // Default copy constructor
68 path_view(const path_view &) = default;
69 // Default move constructor
70 path_view(path_view &&o) noexcept = default;
71 // Default copy assignment
72 path_view &operator=(const path_view &p) = default;
73 // Default move assignment
74 path_view &operator=(path_view &&p) noexcept = default;
75
76 // Swap the view with another
77 constexpr void swap(path_view &o) noexcept;
78
79 // True if empty
80 constexpr bool empty() const noexcept;
81
82 // Exactly the same as for filesystem::path
83 constexpr bool has_root_path() const noexcept;
84 constexpr bool has_root_name() const noexcept;
85 constexpr bool has_root_directory() const noexcept;
86 constexpr bool has_relative_path() const noexcept;
87 constexpr bool has_parent_path() const noexcept;

```

```

88  constexpr bool has_filename() const noexcept;
89  constexpr bool has_stem() const noexcept;
90  constexpr bool has_extension() const noexcept;
91  constexpr bool is_absolute() const noexcept;
92  constexpr bool is_relative() const noexcept;
93
94  // Adjusts the end of this view to match the final separator, same as filesystem::path
95  constexpr void remove_filename() noexcept;
96
97  // Returns the size of the view in characters, same as filesystem::path::native().size()
98  constexpr size_t native_size() const noexcept;
99
100 // Exactly the same as for filesystem::path, but returning a slice of this view
101 constexpr path_view root_name() const noexcept;
102 constexpr path_view root_directory() const noexcept;
103 constexpr path_view root_path() const noexcept;
104 constexpr path_view relative_path() const noexcept;
105 constexpr path_view parent_path() const noexcept;
106 constexpr path_view filename() const noexcept;
107 constexpr path_view stem() const noexcept;
108 constexpr path_view extension() const noexcept;
109
110 // Return the path view as a filesystem path.
111 filesystem::path path() const;
112
113 /*! Compares the two string views via the view's 'compare()' which in turn calls
114 'traits::compare()'. Be aware that if the path views do not view the same underlying
115 representation, a UTF based comparison will occur rather than a 'memcmp()' of
116 the raw bytes.
117 */
118 constexpr int compare(const path_view &p) const noexcept;
119 constexpr int compare(const char *s) const noexcept;
120 constexpr int compare(string_view str) const noexcept;
121 #ifdef _WIN32
122     constexpr int compare(const wchar_t *s) const noexcept;
123     constexpr int compare(wstring_view str) const noexcept;
124 #endif
125
126 // iterator is the same as const_iterator
127 constexpr iterator begin() const;
128 constexpr iterator end() const;
129
130 // Instantiate from a 'path_view' to get a zero terminated path suitable for feeding to the kernel
131 struct c_str;
132 friend struct c_str;
133 };
134
135 // Usual free comparison functions
136 inline constexpr bool operator==(path_view x, path_view y) noexcept;
137 inline constexpr bool operator!=(path_view x, path_view y) noexcept;
138 inline constexpr bool operator<(path_view x, path_view y) noexcept;
139 inline constexpr bool operator>(path_view x, path_view y) noexcept;
140 inline constexpr bool operator<=(path_view x, path_view y) noexcept;
141 inline constexpr bool operator>=(path_view x, path_view y) noexcept;
142 inline std::ostream &operator<<(std::ostream &s, const path_view &v);

```

There is a child helper struct which takes in a path view, and decides whether the path view needs to be copied onto the stack due to needing zero termination and/or UTF conversion, or whether the original collection of bytes can be passed through without copying.

```

1  struct c_str
2  {
3      // Maximum stack buffer size on this platform
4      #ifdef _WIN32
5          static constexpr size_t stack_buffer_size = 32768;
6      #elif defined(PATH_MAX)
7          static constexpr size_t stack_buffer_size = PATH_MAX;
8      #else
9          static constexpr size_t stack_buffer_size = 1024;
10     #endif
11
12     // Number of characters, excluding zero terminating char, at buffer
13     // Some platforms e.g. Windows can take sized input path buffers, and thus
14     // we can avoid a memory copy to implement null termination on those platforms.
15     uint16_t length{0};
16
17     // A pointer to a native platform format file system path
18     const filesystem::path::value_type *buffer{nullptr};
19
20     c_str(const path_view &view) noexcept;
21     ~c_str();
22     c_str(const c_str &) = delete;
23     c_str(c_str &&) = delete;
24     c_str &operator=(const c_str &) = delete;
25     c_str &operator=(c_str &&) = delete;
26
27     private:
28         // Flag indicating if buffer was malloced
29         bool _buffer_needs_freeing;
30
31         // Compilers don't actually allocate this on the stack if it can be
32         // statically proven to never be used
33         filesystem::path::value_type _buffer[stack_buffer_size]{};
34     };

```

The use idiom would be as follows:

```

1  int open_file(path_view path)
2  {
3      // I am on POSIX which requires zero terminated char filesystem paths.
4      // So here if the character after the end of the view is zero, and the view
5      // refers to char* data, we can use it directly without memory copying.
6      path_view::c_str p(path);
7      return ::open(p.buffer, O_RDONLY);
8  }

```

You will surely note the requirement that the character after the path view is legal to read. In this regard, path views are different to string views.

4 Design decisions, guidelines and rationale

There are a number of non-obvious design decisions in the proposed path view object. These decisions were taken after a great deal of empirical trial and error with ‘more obvious’ designs, where those designs were found wanting in various ways. The author believes that the current set of tradeoffs is the ideal set.

The design imperatives for an allocating `std::filesystem::path` are not those for a non-allocating `std::filesystem::path_view`. A ‘handy feature’ of an allocating path object is that it must always copy its input into its allocation. If it is allocating memory and copying in any case, performing an implicit conversion of a native narrow input encoding to say a native wide encoding seems like a reasonable design choice, given the relative cost of the other overheads.

In the case of a path view however, we are trying very hard to not copy memory. If the local platform uses the same narrow or wide input encoding as the source backing the view, and the path view is already terminated by a null character where that is relevant on the local platform, no copying is required. The source backing is used unmodified, bytes are passed through as-is. Only if necessary, a copy and/or conversion of the input onto the stack is performed into whatever format the local platform requires.

One might argue that in the case of `std::filesystem::path`, we might reuse the path across multiple calls, and thus the path view approach of just in time copying per syscall is wasteful on those platforms. However it is exceedingly rare to open the same file more than once, and anyone caring strongly about performance will simply modify their program to use the same native encoding and null termination as the platform.

The next argument is usually one of the form that paths get commonly reused with just the leafname modified, and therefore path’s approach is more efficient as only the leafname gets converted per iteration. I would counter that this proposed path view object comes from [\[P1031\]](#) *Low level file i/o library* where using absolute paths is bad form: you use a `path_handle` to indicate the base directory and supply a path view for the leafname – this is *far* more efficient than any absolute path based mechanism as it avoids the kernel having to traverse the filesystem hierarchy, typically taking a read lock on each inode in the absolute path.

4.1 Why on Microsoft Windows interpret chars as UTF-8 when `std::filesystem::path` interprets chars as ASCII?

`std::filesystem` came originally from Boost.Filesystem, which in turn underwent three major revisions during the Boost peer review as it was such a lively debate. During those reviews, it was considered very important that paths were passed through, unmodified, to the system API. There are very good reasons for this, mainly that filesystems, for the most part, treat filenames as a bunch of bytes without interpreting them as anything. So any character reencoding could cause a path entered via copy-and-paste from the user to be unopenable, unless the bytes were passed through exactly.

This is a laudable aim, and it is preserved in this path view proposal. Unfortunately it has a most unfortunate side effect: on Microsoft Windows, `std::filesystem::path` when supplied with `char`

not `wchar_t`, is considered to be in *ANSI* encoding. This is because the `char` accepting syscalls on Microsoft Windows consume ANSI for compatibility with Windows 3.1, and they simply thunk through to the UTF-16 accepting syscall after allocating a buffer and copying the input bytes into shorts. Therefore on Microsoft Windows, `std::filesystem::path` duly expands `char` input into its internal UTF-16 `wchar_t` storage via direct casting. It does **not** perform a UTF-8 to UTF-16 conversion.

Unfortunately any Microsoft Windows IDE or text editor that I have used recently defaults to creating C++ source files in UTF-8¹, exactly the same as on every other major platform including Linux and MacOS. This in turn means that source code with a `char` string literal such as `"UTF♠stringΩliteral"` makes a UTF-8 `char` string, **not** an ANSI `char` string, which is consistent across all the major platforms. Thus, `std::filesystem::path`'s behaviour on Microsoft Windows is quite surprising: your portable program will not work. What works on all the other platforms, without issue, does not work on Microsoft Windows, for no obvious reason to the uninitiated.

This author can only speak from his own personal experience, but what he has found over many years of practice in writing portable code based on `std::filesystem::path` is that one ends up inevitably using preprocessor macros to emit `L"UTF♠stringΩliteral"` when `_WIN32` and `_UNICODE` are macro defined, and otherwise emit `"UTF♠stringΩliteral"`. The reason is simple: the same string literal, with merely a `L` or not prefix, works identically on all platforms, no locale induced surprises, because we know that string literals in UTF source code will be in *some* UTF-x format. The side effect is spamming your 'portable' program code with string literal wrapper macros as if we were still writing for MFC, and/or `#if defined(_WIN32) && defined(_UNICODE)` all over your code. I do not find this welcome.

I appreciate that outside of North American English, string literals containing non-ANSI characters are very rare. Yet most of the world's C++ programmers do not live in North America, nor speak North American English with its 7-bit clean character set. This stuff is an unnecessary and entirely avoidable pain for a lot of users out there, one which can be made to go away if `char` string literals mean UTF-8 on Microsoft Windows.

I also appreciate that [P0882] *User-defined Literals for std::filesystem::path* will take much of this sort of pain away, and some future Unicode string support for C++ from SG16 will no doubt do better again. But as those are not yet approved additions to the standard, I propose that when `char` strings are supplied as a path string literal, and if and only if a conversion is needed, that we interpret those chars as UTF-8.

I know that this is a breaking change from `std::filesystem::path`, but I would argue that `std::filesystem::path` needs to be similarly changed. UTF-8 source code is very, very commonplace now, much more so than even a few years ago, and it is extremely likely that almost all new C++ written will be in UTF-8. So best to change `std::filesystem::path` appropriately, and if that is too great a breaking change, then these proposed path views are 'fixed' instead.

¹For example, in Microsoft Visual Studio 2017 if you enter characters into source code which cannot be represented by ANSI, when you hit save the IDE will offer to save your file as 'Unicode'. Hitting OK saves the file in UTF-8 with BOM, and the MSVC compiler will then perceive the source code as in UTF-8, exactly as on all other major platforms.

4.2 Requiring legality of read of character after end of view

The reason for this is obvious: POSIX and Win32 syscalls consume zero terminated strings as path input, so we need to probe the character after the path view ends to see if it is zero, because if it is not, then we will need to copy the path view onto the stack in order to zero terminate it.

The question is whether this is dangerous or not. `string_view` does not do this, but then string views have a much wider set of use cases, including encapsulating a 4Kb page returned by `mmap()` where reading the byte immediately after the end of the view would mean a segmentation fault.

Path views do not suffer from that problem. One knows that they represent a path on the file system, and are very likely to be constructed from a source whose representation of a file system path will not vary by much. They are therefore highly unlikely to not be zero terminated *at some point* later on, as operations on path views only ever produce sub-views of some original path, and cannot escape the bounds of the original path. Path string literals are safe, by definition. Even a deserialised path from storage is highly likely to always be zero terminated. Because of the much more limited set of use cases for path views, I believe that this requirement is safe.

There is the separate argument that deviating requirements from `string_view` is unhelpful by confusing the user base, and will produce buggy code. Yet I cannot think of a single non-contrived use case where the legality of reading the character after the end of a path view is problematic, including tripping any static analysis tools or sanitisers. I welcome non-contrived examples of where this is dangerous.

I appreciate that if standardised, most in the C++ user base will not know of this requirement and will write code not taking this requirement into account. I would argue that it will be very unusual for the ignorant to become surprised by this requirement.

4.3 Fixed use of stack in `struct c_str`

Firstly, note that the compiler elides completely the fixed stack buffer for zero termination and UTF conversions caused by instantiating `struct c_str` if the compiler can prove that it will never be used. So if you supply native format, zero terminated input, to the path view constructor, the compiler should spot that the temporary stack buffer is never used, and thus eliminate it. This ought to be the case most of the time.

Secondly, the fixed stack buffer tends to get allocated just before a syscall, and released just after that syscall. Stack cache locality is therefore generally unaffected, and the fixed stack buffer does not remain allocated for long.

64Kb on Microsoft Windows systems may seem excessive, but it is highly unlikely to be a problem in practice as Windows has ample default stack address space reservations, and these allocations never recurse. Of the major POSIX implementations, Linux would potentially allocate 4Kb, MacOS 1Kb, FreeBSD 1Kb onto the stack as those are the settings for their `PATH_MAX` macros.

For those Linux implementations running on embedded systems where 4Kb stack allocations would be unwise, we do provide for the ability to internally use `malloc()` to create storage for the temporary buffer which is freed on `struct c_str`'s destruction. One could, of course, decide that on Microsoft

Windows 8Kb of stack is enough, and paths larger than that go to `malloc()`. Again, I would stress that the programmer can be careful to never send a non-zero terminated string in as a path, and thus completely eliminate the use of temporary buffers on an embedded Linux solution. In any case, path views are considerably less heavy on free RAM than `std::filesystem::path`.

4.4 Why is only char input supported, except on Microsoft Windows?

Unlike `std::filesystem::path` which accepts (i) native narrow encoding (ii) native wide encoding (iii) UTF-8 (iv) UTF-16 and (v) UTF-32 format input (all of which have an implied encoding conversion to internal native format, if necessary), path views need to discourage unnecessary reencoding, as due to it not being cached anywhere, it is inefficient to use an input format not equal to the native format. The programmer must be encouraged to only supply input in encodings which do not perform hidden memory copies and allocations, otherwise the whole point of using a path view is made moot.

On every major platform bar one, the native format is `char`. The only major platform where the native format is not `char` is Microsoft Windows.

Thus, most programs will use `char`, as it is near universal. We do not wish to damage the portability of such programs. We therefore also accept `char` on Microsoft Windows, despite that `wchar_t` is the native format.

Portable code which uses char path strings will therefore be inefficient on Microsoft Windows, but that is no different to the present situation with `std::filesystem::path`. If, at some future point, Microsoft decides to convert their native encoding to UTF-8, we can remove the inefficiency on their platform.

5 Technical specifications

No Technical Specifications are involved in this proposal.

6 Frequently asked questions

6.1 Does this mean that all APIs consuming `std::filesystem::path` ought to now consume `std::filesystem::path_view` instead?

Most of the time, perhaps almost always, yes. `std::filesystem::path_view` implicitly constructs from strings, paths and string literals. Anywhere you are currently consuming `std::filesystem::path` as a parameter, you can start using `std::filesystem::path_view` instead if this proposal is approved. It would remain the case that where a function is returning a new path, `std::filesystem::path` is the correct choice. So inputs would be mostly path views, outputs would be paths.

This author has replaced paths with path views in an existing piece of complex path decomposition and recomposition, and apart from a few minor source code changes to fix lifetime issues, the code

compiled and worked unchanged. Path views are mostly a drop-in replacement for paths, except for when one is creating wholly new paths.

Incidentally, performance of that code improved by approximately twenty fold (20x).

7 Acknowledgements

My thanks to Nicol Bolas and Bengt Gustafsson for their feedback upon this proposal.

8 References

- [P0882] Yonggang Li
User-defined Literals for `std::filesystem::path`
<https://wg21.link/P0882>
- [P1031] Douglas, Niall
Low level file i/o library
<https://wg21.link/P1031>